

深度學習專題報告

Natural Language Processing with Disaster Tweets

組別： 第七組

組員：

- B1228012 游承叡 AI 2
- B1228026 蔡智翔 AI 2
- B1228027 黃子宸 AI 2
- B1228036 郭彥均 AI 2

一、專題概述

1. 研究目標

用自然語言處理技術開發一個能夠自動判斷推特推文是否與真實災難相關的分類系統。這是一個二元分類問題，需要將推文分類為災難相關（標籤 1）或非災難相關（標籤 0）。

2. 技術挑戰

- 語言歧義性： 需要區分真實災難描述與譬喻性表達
- 語境理解： 相同詞彙在不同語境下可能有不同含義
- 資料不平衡： 災難推文與非災難推文的數量分布不均

3. 範例說明

- 非災難推文： "I'm on fire tonight! Winning all my poker games!"
 - 此推文使用"on fire"作為譬喻表達，實際與災難無關（標籤 0）
- 災難推文： "Forest fire near La Ronge Sask. Canada. Residents asked to evacuate immediately."
 - 此推文描述真實的森林火災事件（標籤 1）

二、資料長相

1. 資料集構成

資料集包含三個檔案：

檔案名稱	內容描述	資料筆數
------	------	------

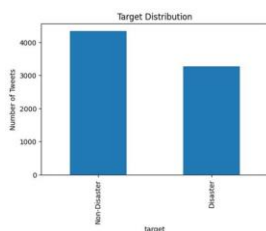
train.csv	訓練資料，包含推文內容及標籤	7613 筆
test.csv	測試資料，不含標籤	3263 筆
sample_submission.csv	Kaggle 提交格式範例	-

2. 資料欄位說明

欄位名稱	資料類型	描述	備註
id	整數	推文唯一識別碼	所有資料集都有
text	字串	推文內容	主要分析對象
location	字串	推文發布地點	可為空值
keyword	字串	推文關鍵詞	可為空值
target	整數	災難標籤（0 或 1）	僅訓練資料有

三、資料分析

1. 類別分布分析



類別	推文數量	比例
災難推文	3,271 筆	約 43%
非災難推文	4,342 筆	約 57%

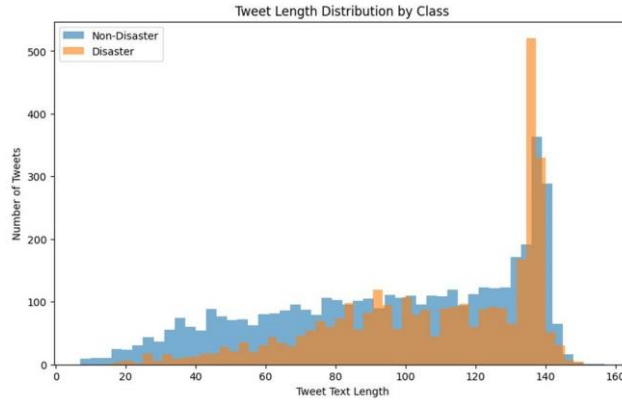
類別不平衡問題：

- 非災難推文數量明顯多於災難推文
- 這種不平衡可能導致模型偏向預測多數類別

解決方案：

- 使用 `class_weight='balanced'` 參數自動調整類別權重
- 考慮採用 SMOTE（Synthetic Minority Oversampling Technique）等過採樣技術
- 評估指標採用 F1-score 而非單純 Accuracy，更適合不平衡資料集

2. 文字長度分析



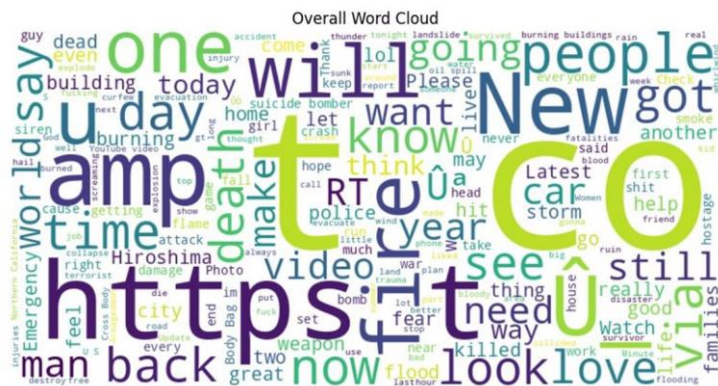
推文長度分布分析顯示：

- 長度集中區間：大部分推文集中在 135-140 字符範圍內
- 類別差異：災難推文平均長度小於非災難推文
- 內容特性推測：
 - 災難推文可能更加簡潔精準，直接描述事實
 - 非災難推文可能較為冗長，包含更多修飾性內容

解決方案：

- 對不同長度推文進行 **padding** 處理與截斷長度設計
- 嘗試將文字長度作為額外特徵納入模型
- 調整 **max_length** 參數為 140 左右以適應推文特性

3. 文字雲分析



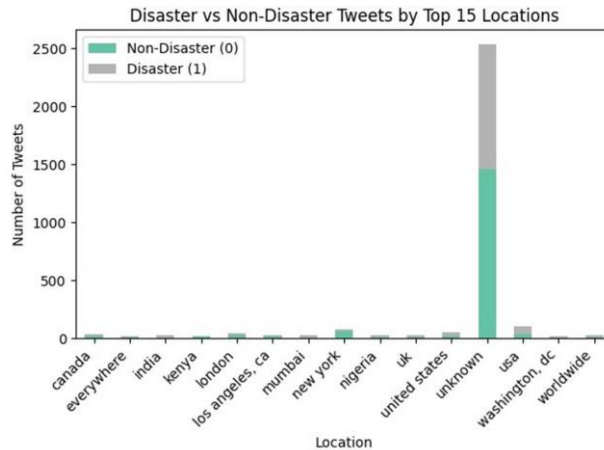
從整體文字雲分析中觀察到：

- 高頻詞彙："one"、"people"、"New"、"got"、"going"等詞彙出現頻率最高
- 災難相關詞彙："fire"、"emergency"、"dead"等災難相關詞彙具有重要特徵價值
- 網址格式問題：推文中常出現"amp"、"https"等網址格式符號，要在處理階段移除

解決方案：

- 使用 TF-IDF、BERT tokenizer 等方法強化語意表徵
- 建立災難相關詞彙字典作為重要關鍵詞特徵
- 清理 URL、特殊符號等雜訊資料

4. 地區分析



根據地區分析圖表發現：

- 未知地點問題：約有 2,500 筆推文的地理位置為未知（unknown）佔總數的大多數
- 災難推文地理分布：在有明確地點的推文中，India、Mumbai、United States、USA 等四個地區的災難推文比例較高
- 資訊收集傾向：特定城市地點中存在災難資訊發布的潛在傾向
- 資料不平衡問題：未知地點資料比例過高，可能影響地理特徵的有效利用

解決方案：

- 由於地點資訊缺失嚴重，建議在模型訓練時降低地點特徵的權重，或考慮使用其他間接地理資訊。

四、預期的實作方法

1. 傳統序列模型 - BiLSTM

為了掌握序列資訊與語意依賴，首先採用 BiLSTM（Bidirectional Long Short-Term Memory）模型作為 RNN 的加強版，觀察其在處理推文文字時的分類表現。

2. 語意強化模型 - BERT 系列

導入 BERT (Bidirectional Encoder Representations from Transformers) 這類預訓練模型。BERT 能更有效捕捉上下文語意，尤其適合處理推文中常見的非正式用語與片語。這也是 Kaggle 上主流的選擇。

3. 大型語言模型 (LLM) - DeepSeek

為了進一步提升模型的理解力與泛化能力，計畫應用 LLM，例如 DeepSeek。這類模型已具備強大的語言理解能力，能更貼近人類語言表達的細節與邏輯。DeepSeek 也提供了相對容易的實作介面，便於我們快速實驗其在特定任務上的效能。

4. 模型融合 - DeBERTa+RoBERTa

計畫採用集成學習的方式，結合 DeBERTa 和 RoBERTa 兩個強大的預訓練模型。DeBERTa 具備改進的注意力機制和位置編碼，而 RoBERTa 則是 BERT 的優化版本，擁有更好的訓練策略。透過融合這兩個模型的預測結果，我們期望能夠取長補短，獲得更穩定且準確的分類表現，這在競賽中往往能取得更好的成績。

五、BiLSTM 模型實作與結果

1. 模型介紹

BiLSTM (Bidirectional Long Short-Term Memory) 是一種雙向長短期記憶網路，能夠同時考慮前向與後向序列的上下文資訊，提升語意理解能力。相較於傳統的單向 LSTM，BiLSTM 在自然語言處理任務中表現更為出色，特別適合文本分類和情感分析等應用。

模型優勢

- 雙向語意理解：同時考慮前向與後向序列的上下文資訊，提升語意理解能力
- 適合處理自然語言任務：如推文分類、情感分析等
- 語境完整性：考慮上下文雙向語意→適理解句子整體語意
- 反諷識別能力：對於推文中反諷或否定語氣特別有幫助

模型挑戰

- 訓練時間較長：訓練時間比單向 LSTM 長
- 過擬合風險：需注意過擬合（加入 Dropout）
- 計算複雜度高：相較於簡單模型需要更多計算資源

2. 資料前處理策略

文字清理流程

- 基本清理：將文字轉小寫、小心清除網址、標籤、表情符號與標點符號
- 失效詞處理：刪除中文失效詞（stopwords）
- 詞形還原：使用 WordNetLemmatizer
- 分詞處理：分詞使用 nltk.word_tokenize

資料切分方式

- 訓練驗證比例：train_test_split(test_size=0.2)：80% 訓練集、20% 驗證集
- 隨機種子固定：random_state=42：固定隨機種子，確保結果可重置

當下閱讀

- 統計詞彙：統計前一萬個最常見的詞彙，建立對應索引
- 特殊標記：
 - <PAD>(0)：填充標記
 - <UNK>(1)：未知詞標記
- 序列長度：max_len=50：最大長度，不足補零、過長截斷

3. 模型架構設計

網路結構

- Embedding 層：詞嵌入層，維度 128，含 padding_idx=0
- BiLSTM 層：群體 LSTM，隱藏層維度 128
- 線性層：輸出層，輸出一個分數，用於二元分類
- 損失函數：BCEWithLogitsLoss()

訓練參數設定

- 優化器：Optimizer=Adam

- 學習率：learning_rate=1e-3
- 正規化：Dropout(p=0.5)：防止過移植
- 早停機制：EarlyStopping(patience=3)：連續 3 次未進步即停止
- 批次大小：batch_size=32
- 訓練輪數：epochs=10

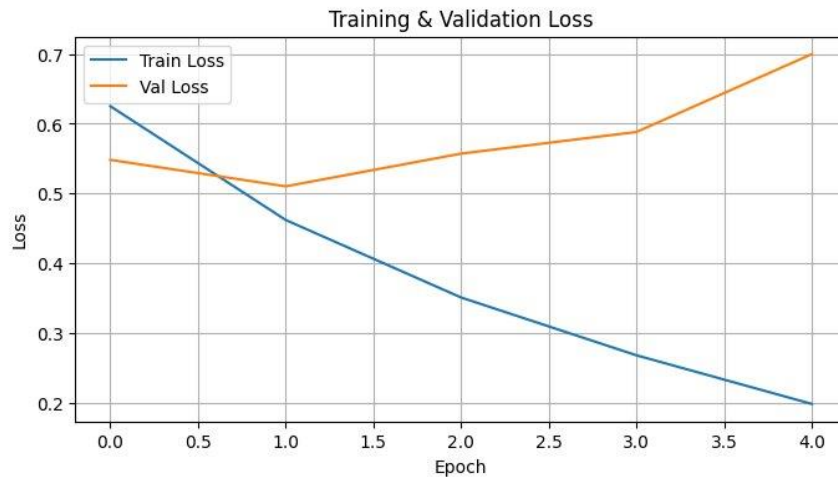
4. 訓練結果分析

Final Evaluation Report:					
	precision	recall	f1-score	support	
0.0	0.78	0.84	0.80	874	
1.0	0.75	0.67	0.71	649	
accuracy			0.77	1523	
macro avg	0.76	0.75	0.76	1523	
weighted avg	0.77	0.77	0.76	1523	

評估指標	類別 0 (非災難)	類別 1 (災難)	整體表現
Precision	0.78	0.75	-
Recall	0.84	0.67	-
F1-Score	0.80	0.71	-
Support	874	649	1523

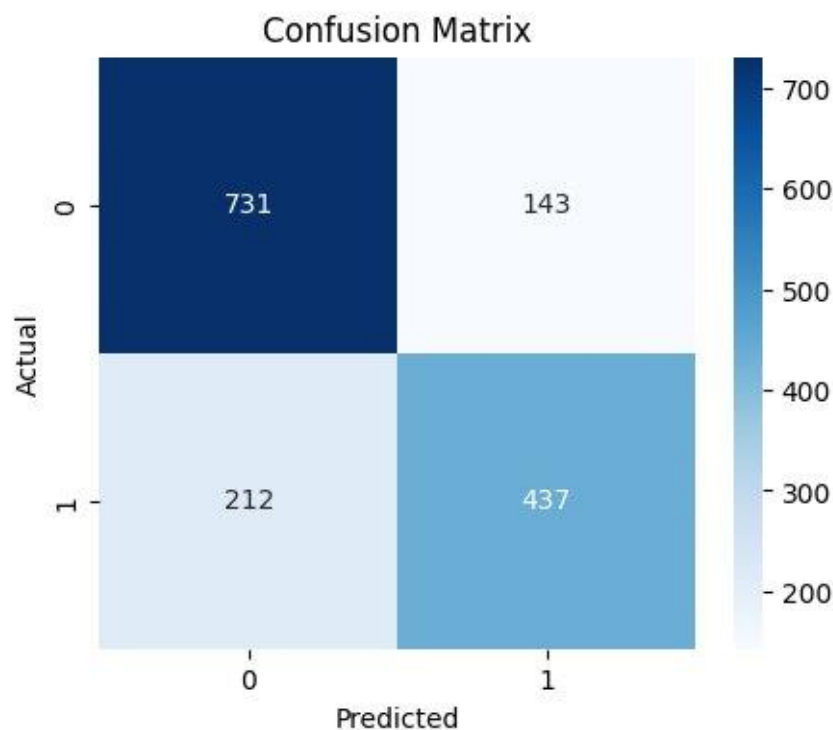
整體指標：

- **Accuracy**：0.77
- **Macro Average**：Precision=0.76, Recall=0.75, F1=0.76
- **Weighted Average**：Precision=0.77, Recall=0.77, F1=0.76



模型行為深度分析

- 類別 0（非災難）表現較佳：recall 較高（0.84），類別 0 的 recall 較高，顯示模型對類別 0 較敏感
- 類別 1（災難）識別困難：類別 1 的 recall 較低（0.67），顯示模型對類別 0 較敏感
- 模型傾向保守：較常將類別 1 判為類別 0



錯誤分析

- False Positives：143 筆（將非災難誤判為災難）
- False Negatives：212 筆（將災難誤判為非災難）
- 模型偏向：模型偏向保守，較常將類別 1 判為類別 0

訓練過程觀察

- 收斂表現：模型在訓練集上收斂良好
- 過擬合跡象：但驗證損失在第 2 個 epoch 後上升，顯示出過擬合，觸發早停機制
- 早停效果：成功避免過度擬合問題

Kaggle 競賽成果

- 最終排名：839/977



BiLSTM_submission.csv
Complete · 1m ago

0.77137

成績分析與改進空間

這個結果顯示 BiLSTM 模型在災難推文分類任務上具有一定的分類能力，但相較於 Kaggle 排行榜上的其他成績，仍有顯著的改進空間。

當前表現評估

- 成功建立了基礎的分類系統
- 模型具備基本的語意理解能力

實作過程中的發現

- 資料預處理的關鍵性：文本清理和標準化對模型最終表現有顯著影響
- 類別不平衡處理：學會了使用適當的評估指標（F1-score）和類別權重調整
- 過擬合識別與處理：理解了 Dropout 和 EarlyStopping 的重要性
- 評估指標選擇：在不平衡資料集中，單純的準確率並不足以評估模型表現

模型局限性認知

- 語意理解深度有限：BiLSTM 雖然能捕捉序列資訊，但對於複雜語意和隱含意義的理解仍有限制
- 訓練資料依賴性：模型表現很大程度上依賴於訓練資料的品質和數量

- 計算效率考量：相較於簡單模型，BiLSTM 需要更多的計算資源和訓練時間

六、BERT 模型介紹

BERT (Bidirectional Encoder Representations from Transformers) 是由 Google 於 2018 年提出的語言表示模型，其架構以 Transformer 的 Encoder 為基礎。相較於傳統單向語言模型，BERT 採用了雙向 (Bidirectional) 注意力機制，能同時考慮輸入序列中左側與右側的上下文資訊，使模型在理解語句語意時更加全面與精確。

BERT 的訓練可分為兩個階段：

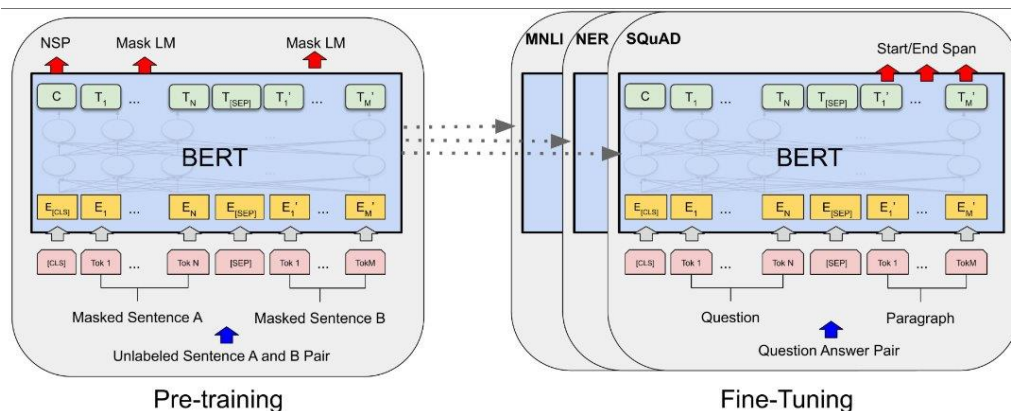
1. 預訓練 (Pre-training)：

在這個階段，BERT 在大規模語料上進行兩項任務的學習：

- **Masked Language Modeling (MLM)**：模型會隨機遮蔽輸入句子中的部分詞語，並學習預測被遮蔽的詞，這有助於模型理解語句中各詞之間的語意關係。
- **Next Sentence Prediction (NSP)**：模型需要判斷兩個輸入句子之間是否具有語意連貫性，此任務可提升模型對文本上下句邏輯關係的理解。

2. 微調 (Fine-tuning)：

在完成預訓練後，BERT 模型會根據特定任務進行微調。我們將任務資料（如災難推文）輸入 BERT，取出每個 token 的語意向量 (representation)，並接上分類器進行災難與非災難推文的二元分類任務。



七、BERT 變體：DistilBERT、DeBERTa、RoBERTa

為探討不同 BERT 架構在災難推文分類任務中的表現差異與實際應用潛力，本研究選用了三種具代表性的 BERT 變體進行實驗，分別為 DistilBERT、RoBERTa 與

DeBERTa。這些模型在原始 BERT 架構基礎上進行了不同程度的優化與改良，適合用於不同計算資源條件與精度要求的情境。

模型名稱	核心設計	優點	缺點	適合應用場景
DistilBERT	為 BERT 的壓縮版本，刪減部分層與 attention heads，保留約 97% 表現，但模型大小僅為原本 40%。	模型參數少、體積小，推論速度快，適合資源有限的裝置。	相較 DeBERTa 與 RoBERTa，其表現略差，特別是在複雜任務上。	適用於行動裝置、低延遲應用等場景。
DeBERTa	引入了解構位置編碼（Disentangled Position Encoding）與增強注意力權重分離設計，使模型能更精準掌握詞與詞之間的語意結構。	表現優異，特別在多數語言任務中均超越 RoBERTa，擁有更強泛化能力。	模型規模大，訓練與推論成本高，較難部署於資源受限環境。	適合對準確率要求較高的任務，如法律文本、醫療分類等。
RoBERTa	改良 BERT 訓練流程，移除 NSP 任務、延長訓練時間，並在更大語料上訓練，進一步提升表現穩定性。	表現穩定、訓練結果重現性佳，整體優於原始 BERT，且訓練成本相對可控。	訓練過程仍需大量計算資源，對硬體需求較高。	通用 NLP 任務的高品質首選模型。

八、DistilBERT 模型實作與結果

1. 資料前處理策略

文字清理流程

- 缺失值填補：將 keyword 與 location 欄位中的缺失值填為空字串，避免在 Tokenizer 處理時產生錯誤。
- 欄位合併：將 text、keyword、location 三欄位合併為一個輸入字串，強化語意整合效果。

資料切分方式

- 訓練驗證比例：使用 `train_test_split(test_size=0.2)` 將資料分為 80% 訓練集與 20% 驗證集。
- 隨機種子固定：設定 `random_state=42`，確保實驗結果可重現。

- 類別平衡切分：使用 `stratify=df['target']` 以保持「災難」與「非災難」的樣本比例一致。

Tokenizer 處理

- 預訓練對應：採用 `DistilBertTokenizerFast.from_pretrained('distilbert-base-uncased')`。
- 自動分詞與截斷：透過 `WordPiece` 分詞機制，並自動進行 `truncation` 與 `padding`。
- 最大長度設定：`max_length=200`，確保輸入不超過模型處理能力。
- 輸入格式：包含 `input_ids`（文字對應的 token ID）與 `attention_mask`（遮蔽 `padding` 區域）。

2. 模型架構設計

網路結構

- 預訓練模型：採用 `distilbert-base-uncased` 作為主體架構。
- 任務設定：針對二分類任務（災難 / 非災難）進行微調（`fine-tuning`）。
- 分類頭：利用 `AutoModelForSequenceClassification` 自動建立分類層，輸出為兩類機率分布。

訓練參數設定

- 優化器：使用 `AdamW`，結合 L2 正則化與學習率衰減，穩定權重更新。
- 正規化係數：`weight_decay=0.01`，抑制過擬合現象。
- 學習率：設定為 `2e-6`，較小值有助於訓練穩定收斂。
- 學習率排程器：採用 `linear` 線性衰減策略，隨訓練進度平滑下降學習率。
- 早停條件：`EarlyStopping(patience=2)`，若連續兩輪驗證結果未改善則提前終止訓練。

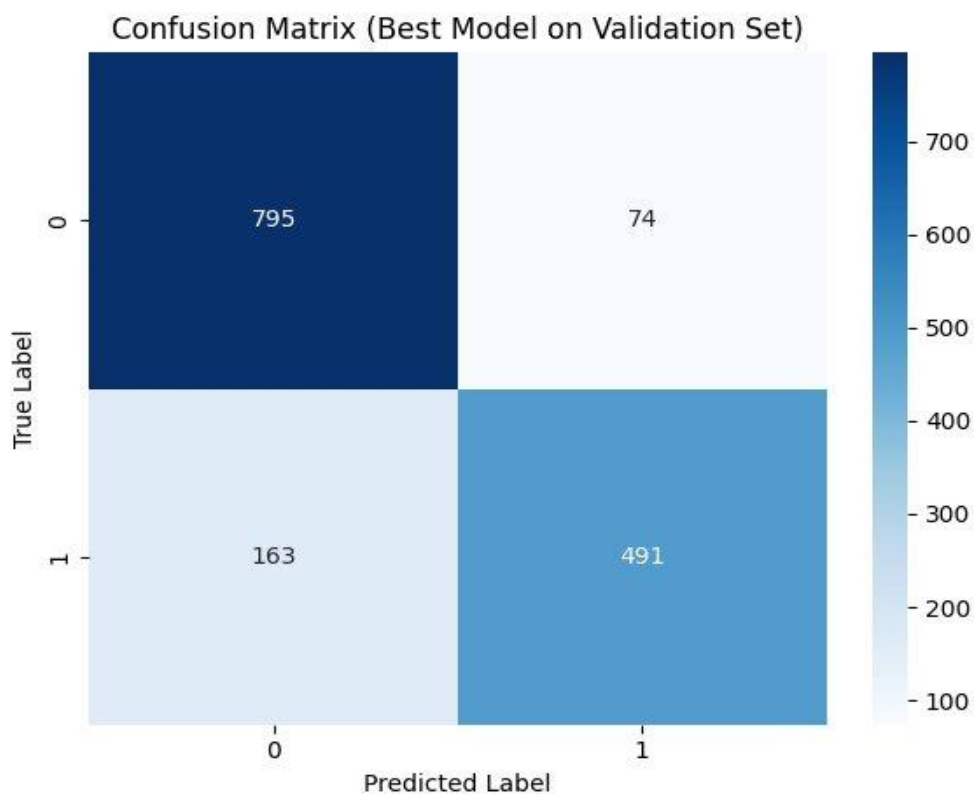
3. 訓練結果分析

評估報告

Final Evaluation Report:					
	precision	recall	f1-score	support	
0	0.84	0.89	0.87	869	
1	0.84	0.78	0.81	654	
accuracy			0.84	1523	
macro avg	0.84	0.84	0.84	1523	
weighted avg	0.84	0.84	0.84	1523	

訓練狀況

- 模型在訓練集上收斂良好，損失穩定下降。然而在第 1 個 epoch 後，驗證損失開始上升，顯示模型出現過擬合傾向，進而觸發 EarlyStopping 機制，避免性能進一步惡化。



分類表現

- 類別 0（非災難）：模型預測準確率較高，僅有 74 筆誤分類為 false positives。

- 類別 1（災難）：存在 163 筆 false negatives，顯示模型對災難文本的辨識相對較弱。
- 模型傾向保守決策，較常將災難類（1）誤判為非災難類（0），導致 recall 偏低。



整體指標

- Precision 與 Recall 均維持在約 0.84，顯示模型預測穩定且泛化能力尚可。
- 類別 0 的 recall 為 0.89，辨識效果優異。
- 類別 1 的 recall 為 0.78，顯示模型在捕捉災難相關文本上仍有進步空間。



DistilBERT_submission.csv
Complete · 15s ago

0.82684

九、DeBERTa 模型實作與結果

1. 資料前處理策略

文字清理流程

- 特殊字元清理：移除網址（URL）、HTML 標籤、表情符號及標點符號，提升文字純淨度。

- 否定詞保留：保留 no、not、never、n't 等否定詞，避免語意誤判。
- 格式統一：將所有文字轉為小寫，並去除多餘空白，增進文字一致性。

資料切分方式

- 訓練驗證比例：train_test_split(test_size=0.2)，劃分為 80% 訓練集與 20% 驗證集。
- 隨機種子固定：設定 random_state=42，確保實驗結果可重現。

Tokenizer 處理

- 預訓練對應：使用 DebertaTokenizer，對應模型為 microsoft/deberta-v3-base。
- 最大長度設定：max_len=128，自動進行 padding 與截斷處理。
- 注意力遮罩：return_attention_mask=True，產生遮罩以輔助語境理解。
- 資料順序打亂：shuffle=True，打亂訓練資料，提升模型泛化能力。

2. 模型架構設計

網路結構

- 主模型架構：microsoft/deberta-v3-base，採用 Disentangled Attention 機制。
- 分類頭建構：透過 AutoModelForSequenceClassification 自動接上分類層。
- 損失函數設定：使用 FocalLoss，針對類別不平衡問題進行處理。

訓練參數設定

- 優化器：使用 AdamW，結合 L2 正則化與學習率衰減。
- 學習率：設定為 2e-5，為 DeBERTa 推薦初始值。
- 正規化係數：weight_decay=0.01，抑制模型過擬合。
- 學習率排程器：採用 CosineScheduleWithWarmup，提供穩定的學習率調整。
- 暖身步數：num_warmup_steps=300，避免訓練初期不穩。
- 早停條件：EarlyStopping(patience=2)，連續 2 次無改善即停止訓練。
- 批次大小：batch_size=32。

- 訓練輪數：epochs=5。

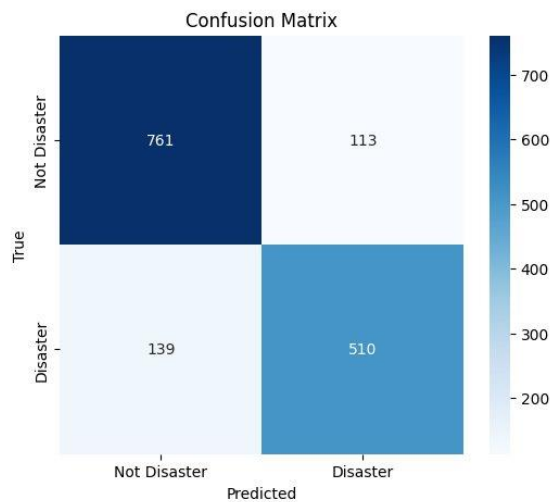
3. 訓練結果分析

評估報告

最終驗證結果 F1 Score: 0.8019 Loss: 0.0261					
	precision	recall	f1-score	support	
0	0.85	0.87	0.86	874	
1	0.82	0.79	0.80	649	
accuracy				0.83	1523
macro avg				0.83	1523
weighted avg				0.83	1523

訓練狀況

- 模型在訓練集上收斂良好，損失穩定下降。然而在第 1 個 epoch 後，驗證損失開始上升，表示模型出現過擬合現象，進而觸發 EarlyStopping 機制，有效避免性能退化。



分類表現

- 類別 0（非災難）：預測相對準確，僅有 113 筆誤分類為 false positives，recall 為 0.87，precision 為 0.85。
- 類別 1（災難）：存在 139 筆 false negatives，recall 為 0.79，precision 為 0.82，預測表現略遜於類別 0。

- 整體誤判傾向與前一模型類似，仍偏向將災難類（1）誤判為非災難類（0），導致 recall 稍低。



整體指標

- 準確率（Accuracy）達 0.83，F1 分數為 0.8019，整體性能穩定。
- Precision 與 Recall 平均約為 0.83，達到可接受水準，適合進一步優化。

✓ DeBERTa_submission.csv
Complete · now

0.82408

十、RoBERTa 模型實作與結果

1. 資料前處理策略

文字清理流程

- 原始文本處理：未進行額外清理，直接將原始文本輸入 tokenizer。
- 基本處理功能：Tokenizer 自行處理標點符號與轉為小寫。

資料切分方式

- 訓練驗證比例：train_test_split(test_size=0.2)，劃分為 80% 訓練集與 20% 驗證集。

- 隨機種子固定：設定 `random_state=42`，確保實驗結果可重現。

Tokenizer 處理

- 預訓練對應：使用 `RobertaTokenizer`，對應模型為 `roberta-base`。
- 空格處理：設定 `add_prefix_space=True`，處理詞首空格問題。
- 最大長度設定：`max_len=128`，自動進行 `padding` 與截斷處理。
- 資料順序打亂：`shuffle=True`，打亂訓練資料，提升模型泛化能力。

2. 模型架構設計

網路結構

- 主模型架構：使用 `roberta-base` 作為文字分類主體架構。
- 分類任務設定：`num_labels=2`，設定為二元分類任務。
- 損失函數設定：使用 `CrossEntropyLoss` 並搭配 `class_weights`，處理類別不平衡。

訓練參數設定

- 優化器：使用 `AdamW`，為含權重衰減之 `Adam` 優化器。
- 學習率：設定為 `1.7e-5`，搭配 L2 正則化。
- 正規化係數：`weight_decay=0.01`，抑制模型過擬合。
- 學習率排程器：設定為 `"linear"`，採線性衰減方式調整學習率。
- 暖身步數：`num_warmup_steps=300`，避免訓練初期不穩定。
- 早停條件：`EarlyStopping(patience=2)`，連續 2 次無改善即停止訓練。
- 批次大小：`batch_size=32`。
- 訓練輪數：`epochs=5`。

3. 訓練結果分析

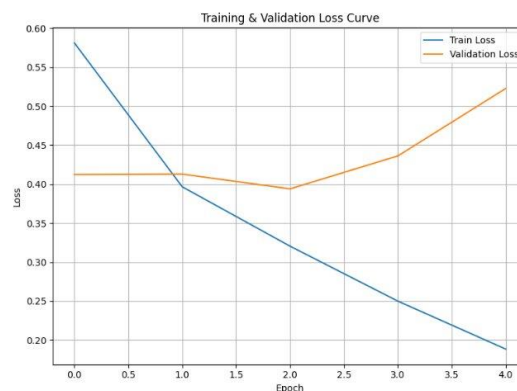
評估報告

Final Evaluation Report:				
	precision	recall	f1-score	support
0	0.85	0.87	0.86	874
1	0.82	0.80	0.81	649
accuracy			0.84	1523
macro avg	0.84	0.83	0.84	1523
weighted avg	0.84	0.84	0.84	1523

根據最終分類報告可得知，模型在兩類別的預測表現如下：

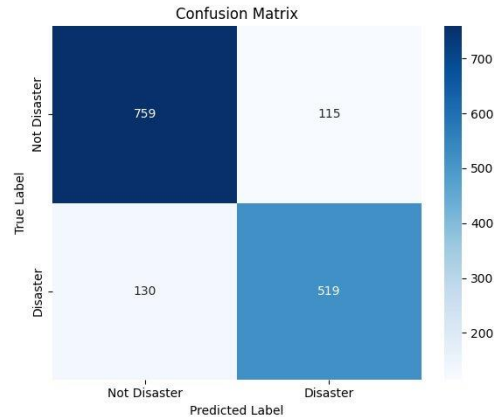
- 類別 0（非災難推文）：Precision 為 0.85，Recall 為 0.87，F1-score 為 0.86，顯示模型在非災難類別的識別準確性與召回率皆具穩定表現。
- 類別 1（災難推文）：Precision 為 0.82，Recall 為 0.80，F1-score 為 0.81，顯示模型對災難推文的預測能力也達到相當水準。
- 整體表現：模型整體 Accuracy 達 0.84，Macro average 與 Weighted average 的 precision、recall 與 F1-score 均約為 0.84，顯示模型對各類別皆有穩健的學習成效，沒有明顯偏倚。

Loss 曲線圖



從訓練與驗證的損失函數變化圖可以觀察到，訓練損失（Train Loss）隨著 epoch 持續下降，顯示模型在學習上具有良好的收斂性。然而，驗證損失（Validation Loss）在第 2 個 epoch 達到最低點後開始上升，可能代表模型逐漸出現過擬合現象。這也印證了早停機制（EarlyStopping）在第 5 個 epoch 停止訓練的必要性，有效防止模型過度學習訓練資料。

混淆矩陣



- 真實為非災難的推文中，模型正確預測了 759 筆，錯誤預測為災難的有 115 筆。
- 真實為災難的推文中，模型正確預測了 519 筆，錯誤預測為非災難的有 130 筆。
- 此結果顯示模型對兩類別皆具有不錯的辨識能力，但在判斷災難推文時相對稍弱，可能仍有部分災難推文因文字特徵模糊或表達不明確而被誤判為非災難。

Kaggle 競賽成果

- 最終排名：161/977



RoBERTa_submission.csv
Complete · now

0.83358

十一、deepseek 模型實作與結果

1. 資料前處理策略

- 移除無效樣本 (在資料集裡 label 是 null 的會被移除)
- 將 text 欄位轉成 string，並標準化空白字元
- 使用自定義 datasets，讀取 Disaster Tweets 資料集(kaggle 提供的)

2. 切分資料

把 train.csv 的資料集，拆分為 80%作為訓練，20%作為驗證資料集並且為訓練資料集打散

3. Tokenizer 的設定

`padding="max_length"`、`truncation=True`：固定長度處理

`max_length=128`：控制輸入長度以降低計算成本

自定義 `tokenize_function`：將文字轉為 `token` 並保留 `label`

4. 模型架構

使用 `AutoModelForSequenceClassification.from_pretrained("deepseek-ai/deepseek-llm-7b-chat", num_labels=2)`

模型架構為 DeepSeek LLM，搭配分類頭進行二元分類

預訓練模型支援 4-bit 量化（使用 `load_in_4bit=True` 降低資源消耗）

5. 訓練策略

使用 `Trainer` 進行訓練與驗證任務

`per_device_train_batch_size=4`：微調大模型建議使用較小 `batch size`

`gradient_accumulation_steps=4`：模擬較大 `batch` 以提升穩定性

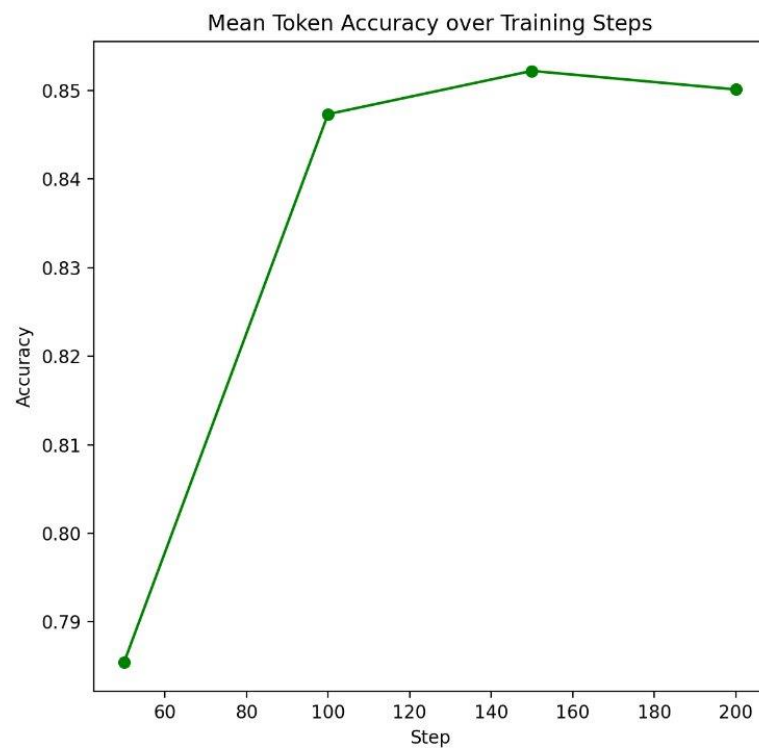
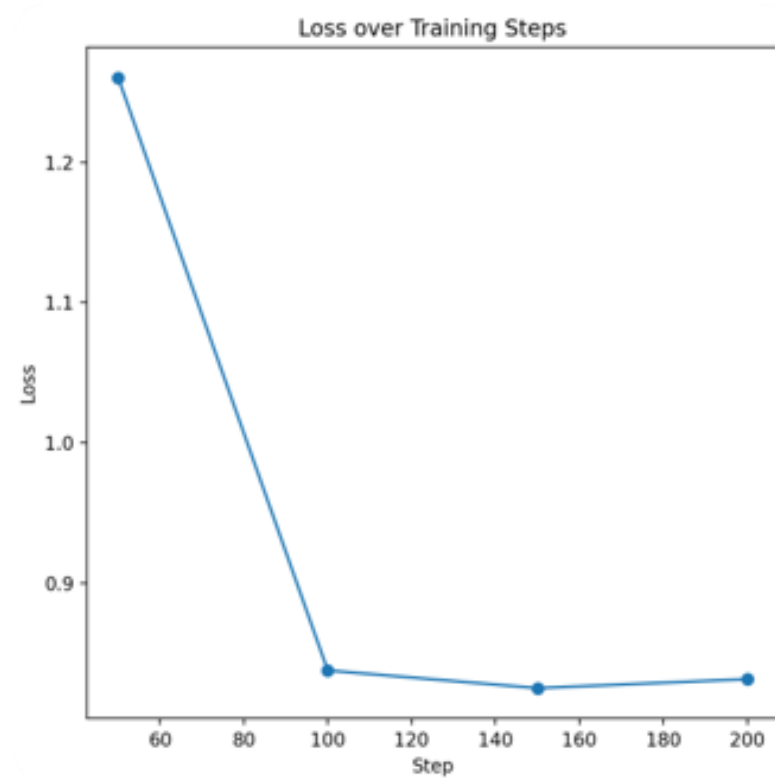
`learning_rate=2e-4`：適用於微調 4-bit 模型的高學習率設定

`lr_scheduler_type="cosine"`：餘弦學習率衰減策略

`early_stopping_patience=2`：若驗證集兩輪未改善即停止訓練

`load_best_model_at_end=True`：保留驗證表現最佳模型

6. 訓練結果



Kaggle 競賽成果

- 最終排名：792/977



十二、RoBERTa 與 DeBERTa 模型融合實作與結果

1. 資料前處理策略

直接提取 `text` 欄位作為輸入文本，`target` 欄位作為標籤
保留測試集的 `id` 欄位用於生成提交文件

2. 切分資料

未進行額外的資料切分，直接使用原始的訓練集進行訓練
簡化處理：使用訓練集同時作為評估集（`eval_dataset=train_dataset`）

3. Tokenizer 的設定

我們調整了 `max length:141`(95%的文本長度在這個範圍內)

4. 模型架構

大致上跟前面的模型設定差不多最大的不同是我們使用餘弦學習率衰減策略
以及更改學習率預熱步數為 `130(total_steps * 0.1)`

5. 為什麼這些調整有助於分數的提升

根據我們文字長度分布圖，我們的文本有 95%的資料小於 141 個字所以實際產生的 Token 數量可能會：

1. 少於 140 個 Token：這種情況比較少見，除非你的「單字」定義非常寬泛，或者文本中包含大量可以合併的字符序列（這在某些語言的 BPE 處理中可能發生，但英文中較少）。
2. 大約等於 140 個 Token：如果你的文本主要由非常常見、能被詞彙表完整覆蓋的單字組成，那麼單字數和 Token 數可能會比較接近。
3. 多於 140 個 Token：這是最常見的情況。因為很多單字會被切分成多個子詞 Token。例如，如果你的 140 個單字中，有 20 個單字每個都被切成了 2 個子詞 Token，那麼光這些單字就會產生 $20 * 2 + (140 - 20) * 1 = 40 + 120 = 160$ 個 Token。

再加上我們發現後面的資大多都是網址跟使用者推特的帳號，因此我們使用自動填充和截斷處時，不會出現截斷重要訊息跟過多的 PADDING 提升我們訓練的準確度

為什麼預熱步數設為 10% (0.1 * total_steps) 是好選擇？

理由	說明
平滑過渡到主學習率	預熱期間學習率線性從 0 漸升到 learning_rate，避免訓練初期不穩。
在穩定與收斂間取得平衡	預熱比例太小（如 1%）會讓學習率升太快，仍有不穩定風險；太大（如 30%）會拖慢學習速度。10% 是廣泛驗證的平衡值。
實驗經驗支持	原始 BERT、RoBERTa、T5 等論文與後續實驗大多使用 warmup=0.1 或相近比例。
可與 Cosine scheduler 搭配	常見 scheduler（如 cosine, linear）在 warmup 結束後會開始平滑地 decay，讓訓練更穩定。

Kaggle 競賽成果

- 最終排名：32/977

 DeBERTaRoBERTa_submission.csv	0.84523
Complete · 1m ago	

十三、實驗設定與結果

實驗一：模型結果比較與分析

實驗設定

指標 / 模型	BiLSTM	DistilBERT	DeBERTa	RoBERTa	DeepSeek	DeBERTa+RoBERTa
Accuracy（準確率）	0.77	0.84	0.83	0.84	0.78	0.89
F1-score（F1 分數）	0.70	0.84	0.80	0.84	x	0.86
Precision（精確率）	0.77	0.84	0.83	0.84	x	0.87

Recall（召回率）	0.75	0.84	0.83	0.84	x	0.87
Loss（驗證損失）	0.52	0.3886	0.0261*	0.38	0.82	0.2869
訓練時間	24 秒 /epoch	50 秒 /epoch	156 秒 /epoch	138 秒 /epoch	2.5 時/epoch *本地端執行	De:159 秒 /epoch Ro:80 秒/epoch *本地端執行
參數量	約 1M	66M	140M	125M	671B（啟用約 37B）	約 265M
模型大小	約 4MB	約 250MB	約 500MB	約 500MB	約 404GB（量 化後）	約 1GB
推論時間	快速	中等	稍慢	稍慢	慢	慢

實驗結果

本實驗旨在探討多種自然語言處理模型於災難推文分類任務中的效能表現，所選模型包括 BiLSTM、DistilBERT、DeBERTa、RoBERTa、DeepSeek，以及融合模型 DeBERTa+RoBERTa。我們觀察其在 Accuracy、F1-score、Precision、Recall、Loss、訓練時間、模型大小與推論效率等面向的差異。

然而，在教師指導下，我們意識到本次實驗仍存在重要限制。由於各模型在資料前處理方式（如分詞器選擇）、訓練策略（如 batch size、learning rate、early stopping 機制）、訓練輪數（epochs）、甚至執行平台（如 Colab vs 本地 GPU）等方面並未完全一致，因此無法作為嚴謹的對照實驗。以下呈現的比較結果，僅為實務操作下的觀察，具參考價值但不具正式實驗結論的嚴謹性。

1. 整體效果最佳：DeBERTa+RoBERTa 融合模型

融合模型整體指標最為出色，準確率達 0.89，F1-score 為 0.86，Precision 與 Recall 均為 0.87，顯示融合不同語言模型能綜合其特性，有效提升分類準確度。雖然推論時間偏慢，但在對準確性要求高的任務中仍具備實用潛力。

2. 訓練與推論效率最佳：BiLSTM

BiLSTM 為傳統神經網路模型，其架構簡單、訓練時間極短（24 秒 /epoch），推論速度也非常快速，模型大小僅 4MB，適合即時處理與資源受限的場景。然而，分類效能較弱，F1-score 僅為 0.70，反映其無法捕捉深層語義特徵。

3. 單一模型中表現最佳：RoBERTa

RoBERTa 在所有單一 Transformer 模型中表現最佳，F1-score 達 0.84，模型規模適中（約 500MB，125M 參數），同時兼顧效能與資源效率，適合部署於中等資源環境中。若需在成本與準確率之間取得平衡，RoBERTa 是較佳選擇。

4. DeepSeek 成效受限於訓練條件

DeepSeek 雖為參數龐大的語言模型（671B 參數，部署後約 404GB），但由於免費資源限制與本地記憶體不足，我們僅完成約 1.5 epoch 的訓練，導致 F1-score 僅為 0.80，未能發揮應有潛力。理論上 DeepSeek 應具有更強的語意理解能力，若能完整訓練應會取得更佳結果。

實驗反思

本次比較提供我們寶貴的實務經驗，然而正如教師所指導，模型效能的比較應建立在相同前提下進行，未來若要進一步驗證模型優劣，應統一：

- 資料前處理流程（tokenizer、padding、lowercasing 等）
- 訓練策略（同樣的 optimizer、batch size、learning rate、epochs）
- 評估條件（相同平台、硬體資源）
- 重複實驗次數以減少偶發誤差

實驗 2：Roberta 參數調整

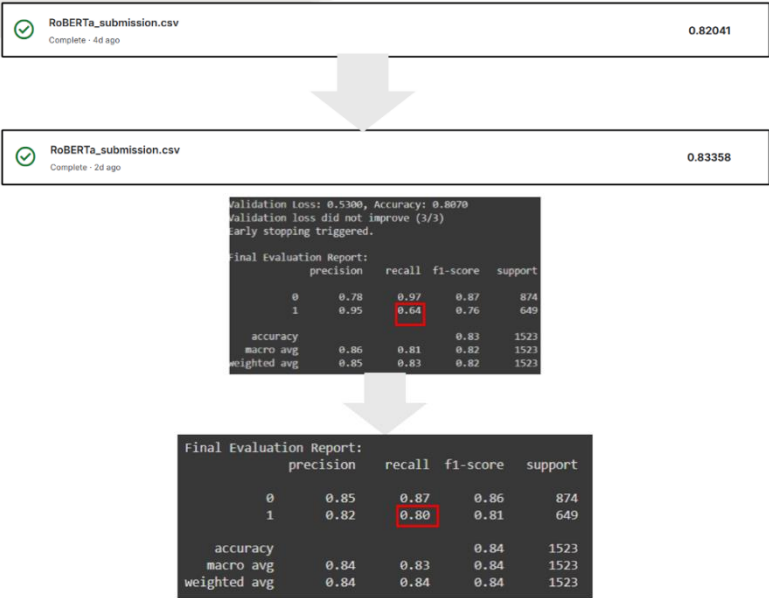
實驗設定

參數	原設定	調整後	調整原因
learning_rate	2e-5	1.7e-5	略微降低學習率可使模型學習過程更穩定，提升其對災難類等語意細緻樣本的學習能力，減少過擬合風險。
batch_size	16	32	增大批次大小有助於梯度穩定，降低每批的隨機變異，有利於模型分類穩定性，但需考慮 GPU 記憶體負載。
epochs	4	5	適度增加訓練次數可給予模型更多學習機會，特別是災難類樣本占比較低的情況，有助於提升召回率。
warmup_steps	500	300	減少 warmup 步數可讓模型更快進入穩定訓練期，避免過長的 warmup 拖延收斂速度。
loss function	CrossEntropyLoss()	CrossEntropyLoss(加上 class weight)	加入 class weight 能針對訓練資料中類別不平衡進行調整，避免模型過度偏向非災難類別，提升對少數類別的辨識能力。

max_length	128	保持	推文篇幅本就偏短，無需拉長序列長度，以避免計算資源浪費。
------------	-----	----	------------------------------

實驗結果

在本次實驗中，我們針對前一階段表現良好的 RoBERTa 模型進行超參數調整，目標是進一步優化模型在災難推文分類任務中的表現，特別是提升模型對少數類別（災難類）的識別能力。調整後模型的 F1-score 從原本的 **0.82041** 提升至 **0.83358**，顯示適當的超參數調整有助於提升整體效能。



1. 經調整後的模型，在 F1-score 提升約 1.3%，顯示多數調整具正面影響，特別是 class weight 與 learning rate 的調整明顯改善模型對少數災難類別的辨識能力。
2. 模型在驗證集中顯示出更穩定的 loss 曲線與最佳的 recall 數值，驗證了調整的合理性。
3. 雖然 warmup 步數減少的影響較小，但仍能稍微加快模型進入學習期，有助提升整體訓練效率。
4. 批次大小的增加使訓練過程更加平穩，但也須確保硬體資源足夠支撐；在本次本地 GPU 訓練環境下運作良好。

本次參數優化實驗顯示，微調訓練策略與超參數設定對模型效能具有明顯影響。透過 class weight 的加入與合適的學習率調整，不僅改善了模型整體表現，也

使模型在處理不平衡資料時更具彈性與穩定性。此實驗驗證了針對特定任務設計參數的重要性，亦提供後續進一步改進與調整的基礎依據。

十四、遭遇問題與對應解法

在本次自然語言處理任務的實作過程中，儘管成功完成多種模型的建置與訓練，但也遇到若干技術與環境層面的挑戰。為維持實驗品質並有效推進專題進度，我們針對每個問題採取了具體應對措施，說明如下：

1. 模型過度擬合

在模型訓練初期觀察到，驗證集的 **F1-score** 指標於初期數個 **epoch** 快速上升，但在達到高點後反而逐步下降，呈現出典型的過度擬合（**overfitting**）現象。這意味著模型在訓練集上表現良好，但無法有效泛化至未見過的資料。

解決方式：為緩解此問題，我們加入了 **EarlyStopping** 機制，透過監控驗證集損失（**validation loss**）或 **F1-score**，在數個 **epoch** 未見顯著改善時提前終止訓練，避免模型持續過度學習訓練集的細節而犧牲泛化能力。

2. AdamW Optimizer 呼叫錯誤

在使用 HuggingFace 的 **transformers** 套件進行模型優化時，原本計畫採用 **AdamW** 優化器。然而在 Google Colab 平台上執行時，發生無法正確呼叫 **transformers.optimization.AdamW** 的錯誤。經查證，問題起因於 Colab 預設安裝的 **transformers** 版本過新，API 結構已與教學資源不同，導致呼叫失敗。

解決方式：為解決相容性問題，我們採取兩種方案：

- 降版 **transformers** 套件至相容版本。
- 改用 **torch.optim.AdamW**，由 PyTorch 官方提供的對應實作替代，成功解決初始化錯誤問題。

3. Kubeflow 訓練平台部署困難

原先規劃嘗試使用 **Kubeflow** 進行模型訓練與管理，藉由 **MLOps** 工具提升訓練流程的自動化與可重現性。然而，實際操作時遇到部署環境相依性錯綜複雜的挑戰，即使多次修改 **Dockerfile** 配置與安裝指令，仍無法成功啟動訓練任務。

解決方式：在評估所耗時間與技術複雜度後，最終決定改回於本地端 **GPU** 環境進行模型訓練。此方式雖較為原始，但更穩定且可控，也能確保順利完成訓練流程。

4. DeepSeek 模型訓練資源受限

在實驗中嘗試引入參數規模極大的 DeepSeek 預訓練語言模型（約 671B 參數，模型大小 404GB），但很快面臨嚴重的資源瓶頸：

- 在 Google Colab 的免費版本中，因 GPU 記憶體限制，無法完整載入模型或進行 batch 訓練。
- 改於本地端 GPU（記憶體僅 8 GB）進行訓練時，訓練過程中頻繁出現記憶體溢出（out-of-memory）錯誤。

解決方式：為避免資源耗盡，我們選擇將訓練次數縮短至 1.5 epoch，僅驗證其初步表現。然而，這也導致該模型的 F1-score 僅達 0.80，明顯低於預期。整體而言，該模型理論潛力高，但訓練條件不利，實際效果無法充分發揮。